

Simulación de Enjambre de Peces en 3D

*Algoritmo Boids e Implementación en Babylon.js con Shaders
Personalizados e Instanciación*

Proyecto P2

Gonzales Rodriguez Jose Alberto

Martinez Barralles Oscar

Mejia Garcia Oswaldo

24 de junio de 2026

1. Introducción

El presente proyecto documenta la implementación de una simulación tridimensional interactiva de un banco o enjambre de peces (*fish school*). Dicha simulación está construida enteramente sobre tecnologías web estándar utilizando HTML5, CSS3, JavaScript moderno (ES6) y la biblioteca de gráficos 3D **Babylon.js** a través de su CDN oficial.

El objetivo del proyecto es recrear el comportamiento colectivo complejo y emergente observable en la naturaleza mediante reglas físicas simples, combinándolo con técnicas avanzadas de computación gráfica de alto rendimiento, tales como el renderizado de mallas instanciadas y sombreadores (shaders) programados directamente en lenguaje GLSL (OpenGL Shading Language).

La simulación consta de más de 150 agentes autónomos navegando por un espacio tridimensional acotado que representa un entorno submarino profundo, completo con iluminación difusa y especular, proyección de sombras, niebla de profundidad y partículas de aire en ascenso.

2. Algoritmo de Enjambres (Boids)

El comportamiento del enjambre se basa en el algoritmo **Boids**, desarrollado por Craig Reynolds en 1986. Cada pez es un agente autónomo (*boi*d) que evalúa su entorno local en cada cuadro (*frame*) y calcula una fuerza de aceleración resultante basándose en tres reglas fundamentales más comportamientos personalizados para el proyecto.

2.1. Reglas Fundamentales

1. **Separación (*Separation*)**: Evita colisionar con compañeros del banco que estén excesivamente cerca. Cada agente aplica una fuerza repulsiva inversamente proporcional a la distancia con los vecinos dentro de un radio de separación determinado.

La fuerza de separación $\vec{F}_{sep}$ para un agente i se define como:

$$\vec{F}_{sep} = \sum_{j \in S_i} \frac{\vec{x}_i - \vec{x}_j}{\|\vec{x}_i - \vec{x}_j\|^2} \quad (1)$$

Donde S_i es el conjunto de agentes vecinos que se encuentran dentro del radio de separación de seguridad y $\vec{x}$ representa la posición tridimensional del agente.

2. **Alineación (*Alignment*)**: Intenta igualar la velocidad y dirección de movimiento de los compañeros de su mismo grupo en el entorno cercano.

La fuerza de alineación $\vec{F}_{aln}$ se calcula como:

$$\vec{F}_{aln} = \left(\frac{1}{|A_i|} \sum_{j \in A_i} \vec{v}_j \right) - \vec{v}_i \quad (2)$$

Donde A_i es el grupo de vecinos dentro del radio de alineación, $\vec{v}_j$ es el vector de velocidad del vecino j , y $\vec{v}_i$ es la velocidad del agente actual.

3. **Cohesión (*Cohesion*)**: Dirige al agente hacia el centro de masa o posición media de los vecinos locales para mantener al banco unido.

La fuerza de cohesión $\vec{F}_{coh}$ se formula como:

$$\vec{F}_{coh} = \left(\frac{1}{C_i} \sum_{j \in C_i} \vec{x}_j \right) - \vec{x}_i \quad (3)$$

Donde C_i representa a los vecinos dentro del radio de cohesión.

2.2. Reglas y Comportamientos Adicionales

Adicionalmente a las tres reglas clásicas, esta simulación incorpora dinámicas avanzadas para simular jerarquías y confinamiento:

- **Seguimiento del Líder (*Leader Follow*)**: Existen tres peces líderes especiales (de color naranja y escala sustancialmente mayor). Los peces ordinarios están divididos en tres grupos, y cada grupo tiene asignado un líder específico al cual tienden a seguir aplicando una fuerza atractiva hacia su posición.

- **Confinamiento por Límites de Caja (*Boundary Confinement*):** Los peces se mueven dentro de un volumen cúbico definido por un límite global. Al aproximarse a los extremos de la caja, el boid experimenta una fuerza restauradora suave dirigida en sentido opuesto al límite sobrepasado, evitando salidas del escenario sin cortes abruptos en el movimiento:

$$F_{bound,x} = -k \cdot (p_x - B) \quad \text{si } p_x > B \quad (4)$$

Donde B representa la frontera del volumen, p_x la componente de posición del boid, y k la constante de restitución.

3. Arquitectura Gráfica y Escena

La escena de Babylon.js se compone de varios elementos estructurados jerárquicamente para lograr una inmersión visual de alta calidad.

3.1. Cámara ArcRotateCamera

Se ha dispuesto una cámara de tipo orbital (`ArcRotateCamera`) configurada con los siguientes parámetros técnicos:

- **Campo de visión (FOV):** 60° (expresado en radianes como $\pi/3$), que ofrece una perspectiva natural del espacio tridimensional.
- **Plano de Corte Cercano (*MinZ*):** 0,1 unidades, evitando el solape o desaparición de geometría al acercarse a la cámara.
- **Plano de Corte Lejano (*MaxZ*):** 3000 unidades, permitiendo visualizar la totalidad del entorno y el plano del fondo marino sin recortes abruptos.
- **Interactividad Integrada:** Permite la rotación libre en los ejes horizontal y vertical (órbita), desplazamiento lateral en el plano focal (pan) utilizando el botón derecho del ratón, y acercamiento gradual (zoom) mediante la rueda del ratón.
- **Resize Automático:** Se suscribe al evento `resize` de la ventana web para ajustar el ratio de aspecto y refrescar el lienzo del motor sin deformación de imagen.

3.2. Iluminación y Sombras

Para dar realismo tridimensional, se combina luz direccional y ambiental:

- **Luz Hemisférica (`HemisphericLight`):** Actúa como iluminación ambiental general. Posee un tono azul claro orientado desde la superficie (`diffuse`) para imitar la dispersión de luz en el agua, y un color de suelo oscuro para representar la penumbra del fondo.
- **Luz Direccional (`DirectionalLight`):** Simula la luz focal del sol penetrando verticalmente la columna de agua. Posee una dirección inclinada e intensidad pulsante suave en función del tiempo para simular la oscilación de la superficie del agua.
- **Generación de Sombras (`Shadow Maps`):** Se crea un mapa de sombras de 1024×1024 píxeles acoplado a la luz direccional. Se emplea sombreado exponencial suavizado (`useBlurExponentialShadowMap`) con un filtro de desenfoque de 16 (`blurKernel`) para lograr sombras realistas y suaves proyectadas sobre los propios peces y sobre el fondo marino.

3.3. Ambiente y Efectos de Postprocesado

- **Niebla Exponencial** (`Scene.FOGMODE_EXP2`): Se implementó una niebla de densidad 0,006 con una coloración azul marina profunda (RGB: 0.01, 0.04, 0.12). Este efecto atenúa los objetos distantes de forma no lineal, emulando con fidelidad la absorción lumínica del agua.
- **Burbujas Ascendentes**: Un sistema de 60 esferas translúcidas con material estándar de baja opacidad y alta especularidad asciende desde el lecho marino. Cada burbuja tiene una velocidad y una frecuencia de oscilación horizontal propia, reapareciendo en la parte inferior al rebasar el límite superior.

4. Sombreadores Personalizados (Custom Shaders)

Para dotar de dinamismo y personalidad estética única a los peces, se prescindió del material genérico y se programó un **Shader Material** personalizado a bajo nivel en GLSL.

4.1. Vertex Shader: Efecto de Ondulación (Wobble)

El vertex shader es el responsable de calcular la posición proyectada de cada vértice. Para evitar animaciones rígidas o deformaciones a nivel de CPU (lo cual degradaría el rendimiento), el movimiento del pez se modela directamente en la GPU aplicando una deformación sinusoidal en su eje *Z* local basándose en su coordenada *X* local y en el tiempo.

El fragmento del código del Vertex Shader encargado de la deformación es:

```
1 // wobble (deformacion en el eje Z local)
2 float freq = 5.0;
3 float amp = 0.28;
4 float t = time * 12.0 + timeOffset;
5
6 // La cabeza (x > 1.0) se mantiene estable, mientras que la cola (x <
   -1.0) ondula
7 float tailFactor = clamp((2.0 - position.x) / 5.0, 0.0, 1.0);
8
9 vec3 deformedPosition = position;
10 deformedPosition.z += sin(position.x * freq - t) * amp * tailFactor;
```

Explicación Física del Efecto:

- El factor `tailFactor` calcula de forma lineal un multiplicador en función de la coordenada *X* del pez (la cual representa el largo del cuerpo de la cabeza a la cola). Si la coordenada *X* es alta (cabeza del pez), el factor se aproxima a 0, impidiendo la deformación del rostro y ojos. Si la coordenada *X* decrece hacia la cola, el factor se incrementa progresivamente hasta llegar a 1,0, logrando que la aleta caudal sea la que oscile con mayor amplitud.
- Para evitar que todos los peces naden al mismo unísono, se introduce la variable instanciada `timeOffset`, que desfasa el valor temporal para cada pez individual de forma aleatoria.

4.2. Fragment Shader: Cel-Shading y Rim Light

El fragment shader determina el color final de cada píxel utilizando una técnica de *Toon Shading* (o sombreado de celdas) combinada con sombreado de bordes oscuros para lograr un efecto visual de estilo dibujo animado (*Stylized Toon*).

1. **Toon Shading de 4 Niveles:** En lugar de aplicar una degradación lineal y suave del color (*Lambertian reflectance*), la iluminación difusa se cuantiza en cuatro intervalos discretos:

```

1   float intensity;
2   if (diffuse > 0.8) {
3       intensity = 1.0;
4   } else if (diffuse > 0.5) {
5       intensity = 0.7;
6   } else if (diffuse > 0.2) {
7       intensity = 0.4;
8   } else {
9       intensity = 0.15;
10  }
11

```

2. **Especularidad Toon:** Se calcula a través del vector medio (*Halfway Vector*) entre la dirección de la cámara y la dirección de la luz. Si el producto escalar supera un umbral estricto (0,6), se añade un brillo blanco sólido instantáneo (*specular highlight*), en lugar del clásico decaimiento de Blinn-Phong.
3. **Rim Lighting / Outline (Silueta):** Para acentuar las siluetas de los peces bajo el agua, se calcula el producto escalar entre la normal del vértice y la dirección de vista del observador (`dot(normal, viewDir)`). A menor coincidencia (bordes del objeto), el factor tiende a cero, multiplicando el color de salida por un tono oscuro que actúa como contorno de tinta negra.

5. Optimización y Rendimiento

Dibujar de forma individual 150 modelos tridimensionales complejos requeriría 150 llamadas de dibujo separadas (*Draw Calls*) enviadas desde el procesador central (CPU) a la tarjeta gráfica (GPU). Esto saturaría el bus del sistema y limitaría gravemente la tasa de cuadros por segundo (FPS). El proyecto soluciona esto mediante dos técnicas cruciales:

5.1. Fusión de Mallas (Merge Meshes)

El pez base se compone de tres geometrías primitivas creadas proceduralmente: un elipsoide estirado para el cuerpo central, un cilindro aplanado para la aleta caudal, y un cilindro orientado para la aleta dorsal.

Antes de realizar copias o instancias, se ejecuta la función `BABYLON.Mesh.MergeMeshes` sobre estas tres mallas, combinándolas de forma permanente en un único objeto geométrico unificado a nivel de memoria RAM. Esto asegura que el motor 3D trate al pez como una única entidad sin sobrecostes de jerarquía de transformación.

5.2. Renderizado por Instancias (InstancedMesh)

Para duplicar el pez fusionado 150 veces en la escena, se hace uso del mecanismo de **Instanciación de Hardware**. A través de `baseFishMesh.createInstance()`, se le indica a la tarjeta gráfica que renderice el mismo búfer de vértices múltiples veces utilizando una única instrucción de renderizado.

Para que cada pez muestre características individuales manteniendo la velocidad de la instanciación, se emplean **Instanced Buffers** (`registerInstancedBuffer`):

- `timeOffset`: Un flotador único por pez que desfasa la onda de nado en el Vertex Shader.
- `customColor`: Un vector tridimensional (`Vector3`) que almacena la componente RGB específica de color de cada pez. De este modo, la GPU puede renderizar peces de colores verde azulado, violeta, celeste y naranja en un solo paso físico de dibujado.

6. Interfaz y Usabilidad

La aplicación web dispone de varios componentes front-end integrados en el HTML utilizando CSS moderno y posicionamiento fijo:

- **Pantalla de Carga (Loading Screen):** Un contenedor superpuesto a pantalla completa con un fondo degradado marino oscuro, una animación CSS oscilante de un pez, y un mensaje de estado. Este se desvanece de manera suave (`transition: opacity 0.8s`) una vez que el motor de Babylon.js ha completado la inicialización de los búferes y shaders.
- **HUD de Título:** Situado en la esquina superior izquierda, muestra el título del simulador y breves indicaciones de control interactivo en formato minimalista con sombras en el texto para garantizar su lectura frente al fondo marino.
- **Panel de Información Técnica:** Situado en la esquina inferior derecha, provee información al usuario acerca de los componentes activos de la escena, como el tipo de cámara (`ArcRotateCamera`), las luces, los materiales, los controles y el bucle de animación.
- **Contador de FPS:** Ubicado en la esquina superior derecha, se actualiza a intervalos de 500 milisegundos consultando `engine.getFps()`, lo que permite auditar en tiempo real la estabilidad y el rendimiento de la GPU (típicamente manteniéndose a 60 FPS estables).